

Rare Event Sampling with a Global Climate Model and AI Emulator

Nishidh Singh (Roll no.- 20221181)
Supervisor: Dr. Bedartha Goswami

20-04-2026

Contents

1	Introduction	1
1.1	Background	
1.2	Motivation	
1.3	Aim and scope	
2	About the model (emulator)	2
2.1	Data	
2.2	Architecture	
2.3	Training	
2.4	Evaluation	
3	The GCM + AI Emulator Setup	5
3.1	Generating spinup state	
3.2	Direct Numerical Simulation (DNS)	
3.3	Resampling with AI Emulator (AI+RES)	
3.4	Results	
3.5	Resampling algorithm	

1 Introduction

1.1 Background

Extreme weather events like floods, landslides, cyclones, heatwaves etc., have an enormous detrimental impact on society, and thus it is important for us to get a reliable estimate of their likelihood of occurring. The problem of estimating their probabilities is an inherently difficult problem, due to the limited samples that we have for them. For example, when using a numerical (physics-based) model, a massive number of simulations (of the order of at least 10^4) will be required to capture events with a return time of 10^4 years. Thus a better alternative could be to use more efficient algorithms for Rare Event Sampling (RES). The broader idea in such algorithms is as follows:

- start with an ensemble of initial conditions (perturbed from some best estimate of initial state);
- instead of letting the simulation run till the end, we stop the simulation at certain timesteps (called *resampling times*);
- at each resampling time, *score* the trajectories based on how likely they are to eventually evolve into the event of interest;
- finally, clone or duplicate the trajectories that are more ‘promising’, and terminate or kill the ones that are not.
- repeat for other resampling times.

Such an algorithm would naturally guide the simulation towards the event of interest.

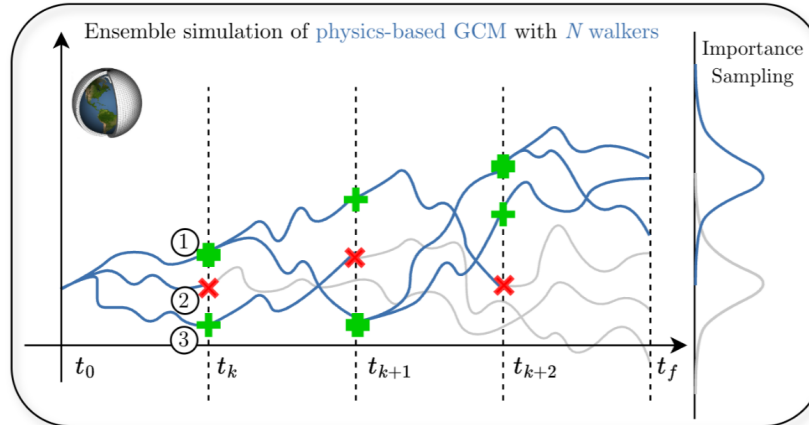


Figure 1: Schematic illustration of Rare Event Sampling [1]

1.2 Motivation

The efficiency or reliability of RES (Rare Event Sampling) is thus entirely dependent on the *scoring function* being used to rank the trajectories. Constructing this scoring function for a high-dimensional and chaotic system such as the climate or weather can be very difficult, and requires often a deep knowledge of the domain, along with a lot of parameter optimization. Approaches that try to use a naive scoring function – say, the ‘persistence’ (current value of target variable) – are likely to fail since, for a highly nonlinear and chaotic system, the current state may be a poor predictor of future state.

At the same time in recent years, machine learning weather prediction models (MLWPs) have emerged as a promising alternative for mid-term weather forecasting, often showing skill comparable to – and for some target variables, even better than – traditional physics-based models. More importantly, they are orders of magnitude quicker to run than traditional physics-based numerical weather prediction models (NWP). Thus, such models may find use in situations where a fast prediction of atmospheric states up to 15–20 days is required.

1.3 Aim and scope

In this project, we address the computational limitations of sampling rare weather events by coupling an aquaplanet Global Climate Model (henceforth referred to as *GCM*), with a neural network trained on long-runs of the same GCM. Our neural network’s architecture is inspired heavily by FuXi-ENS [2], though with some major changes highlighted in the model architecture section.

In our setup, the GCM acts as the physics engine to evolve the trajectories, and the AI emulator serves to score these trajectories (or ‘walkers’). We use the exact resampling scheme described in Amaury et. al [1]. The results of resampling are compared with that of Direct Numerical Simulation (henceforth referred to as *DNS*): this involves, as described earlier, creating an ensemble of initial conditions – each perturbed from some best estimate of initial state – and letting the simulation run till the end without any interruption or resampling. The whole project is an attempt to replicate the methodology described in the same paper [1], but of course with a different GCM and neural network.

2 About the model (emulator)

2.1 Data

We use 260 years of daily average data, generated from long runs of the aquaplanet Global Climate Model (GCM), and an additional 60 years for validation. The training objective is to predict the atmospheric state at time $(t + 1)$ -th day, given the preceding states at (t) -th and $(t - 1)$ -th days. Data is structured on a regular 64×128 grid (latitude $\times$ longitude); this corresponds to a resolution of 2.8125° . Before putting into the network, all input features are channel-wise normalized (z-score).

Type	Variable	Pressure Levels (hPa)
Surface (2D)	Surface Air Temperature	–
Surface (2D)	Surface Air Pressure	–
Upper-Air (3D)	Air Temperature	260, 500, 860
Upper-Air (3D)	Specific Humidity	260, 500, 860
Upper-Air (3D)	Eastward Wind	260, 500, 860
Upper-Air (3D)	Northward Wind	260, 500, 860

Table 1: Input and output variables for model

2.2 Architecture

The architecture has two components:

- **Perturbation model:** this transforms input data into a Gaussian distribution that captures the probabilistic characteristics of the data. The input X is processed through two sequential 2D convolutions. The output is then passed through another two *parallel* 2D convolution projection heads to predict parameters of the multivariate Gaussian latent space: the mean $\mu \in R^{28 \times 64 \times 128}$ and the log-variance $\log \sigma^2 \in R^{28 \times 64 \times 128}$. Now to generate an ensemble of size N , the model simply draws N independent noise tensors from distribution:

$$z^{(n)} = \mu + \sigma \odot \epsilon^{(n)}, \quad \epsilon^{(n)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad n \in \{1, \dots, N\}$$

This (flow dependent) noise is directly added to the original input state to give us N independent perturbed input tensors: $\tilde{X}^{(n)} = X + z^{(n)}$, where $\tilde{X}^{(n)} \in R^{28 \times 64 \times 128}$.

- **Forecasting model:** this module serves as the primary predictive engine. The perturbed input state (variables at times t and $t - 1$) is passed to predict the atmospheric state at $t + 1$. The network takes in the perturbed state tensor $\tilde{X}^{(n)}$. A 2D convolution layer embeds the physical input channels into a latent

representation. The data is then processed through consecutive stages. Each stage applies two sequential Swin Transformer blocks: the first block applies a standard window partition with an 8×8 window size, and the second block applies a shifted window configuration (shift=4). We don't reduce the spatial resolution across all blocks, since our data already is on a lower resolution grid. The final processed sequence is then reshaped back into the 2D spatial grid format. Finally, a standard 2D convolution is used to map the latent features back to the required target channels. This produces the final deterministic forecast: $\hat{X}^{t+1(n)}$.

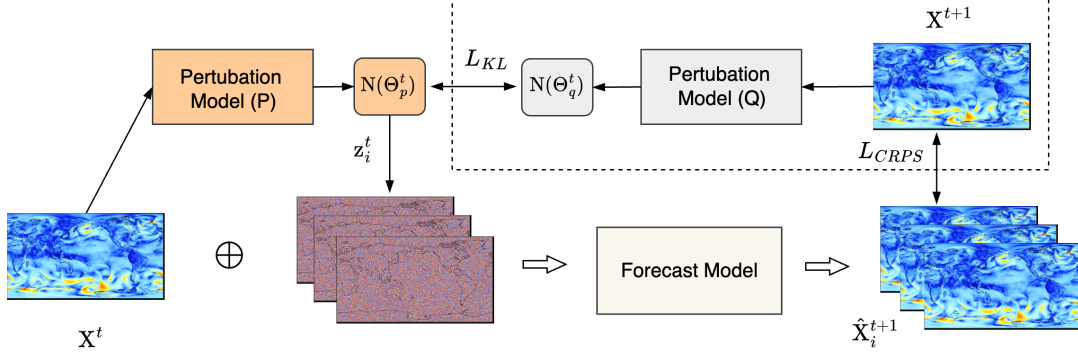


Figure 2: Model architecture

2.3 Training

The training is done in two stages:

- **Pretraining of forecasting model**

In the first stage, the perturbation model is disabled. Only the deterministic model is trained to predict the ground truth state at $t + 1$ given the historical states at t and $t - 1$. This is done for 60 epochs. To account for the decreasing surface area of grid cells near the poles on a regular latitude-longitude grid, the model is optimized using a latitude-weighted L_1 loss function. The loss is:

$$\mathcal{L}_1 = \frac{1}{C \times H \times W} \sum_{c=1}^C \sum_{i=1}^H \sum_{j=1}^W a_i \left| \hat{X}_{c,i,j}^{t+1} - X_{c,i,j}^{t+1} \right| \quad (1)$$

where $C = 14$, $H = 64$, $W = 128$, $\hat{X}^{t+1}$ is the predicted state, and X^{t+1} is the ground truth. The spatial weighting factor a_i at latitude Φ_i is:

$$a_i = \frac{H \cos \Phi_i}{\sum_{k=1}^H \cos \Phi_k} \quad (2)$$

- **Joint training of perturbation and forecasting models**

In the second stage, the pre-trained forecasting model and the perturbation module are jointly trained. During this phase, an ensemble of size $N = 4$ is generated for each forward pass. Here the loss consists of two terms: the Continuous Ranked Probability Score (CRPS) and the Kullback-Leibler (KL) divergence:

$$\mathcal{L}_{Stage2} = CRPS(\hat{\mathbf{X}}^{t+1}, X^{t+1}) + \lambda D_{KL}(\mathcal{N}(\mu, \sigma^2) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I})) \quad (3)$$

where the scaling parameter λ is set to 10^{-4} . This is done for another 30 epochs. The KL divergence is basically a regularization term on the perturbation module's latent space. The CRPS evaluates the probabilistic skill of the N -member ensemble predictions $(\hat{X}^{(1)}, \dots, \hat{X}^{(N)})$ against the deterministic ground truth target X^{t+1} . It is computed as follows:

$$CRPS = \frac{1}{N} \sum_{n=1}^N \left| \hat{X}^{(n)} - X^{t+1} \right| - \frac{1}{2N^2} \sum_{n=1}^N \sum_{m=1}^N \left| \hat{X}^{(n)} - \hat{X}^{(m)} \right| \quad (4)$$

The first term penalizes the mean absolute error of the ensemble members relative to the ground truth. The second term ensures that there is enough spread (or diversity) among the ensemble members.

2.4 Evaluation

The anomaly correlation coefficient (ACC) is calculated using the formula (latitude-weighting used):

$$ACC = \frac{\sum_{i=1}^{N_{lat}} \sum_{j=1}^{N_{lon}} w_i f'_{i,j} a'_{i,j}}{\sqrt{\left(\sum_{i=1}^{N_{lat}} \sum_{j=1}^{N_{lon}} w_i (f'_{i,j})^2\right) \left(\sum_{i=1}^{N_{lat}} \sum_{j=1}^{N_{lon}} w_i (a'_{i,j})^2\right)}}$$

where f' , a' represent respectively the forecast and observation anomalies (calculated w.r.t. climatology).

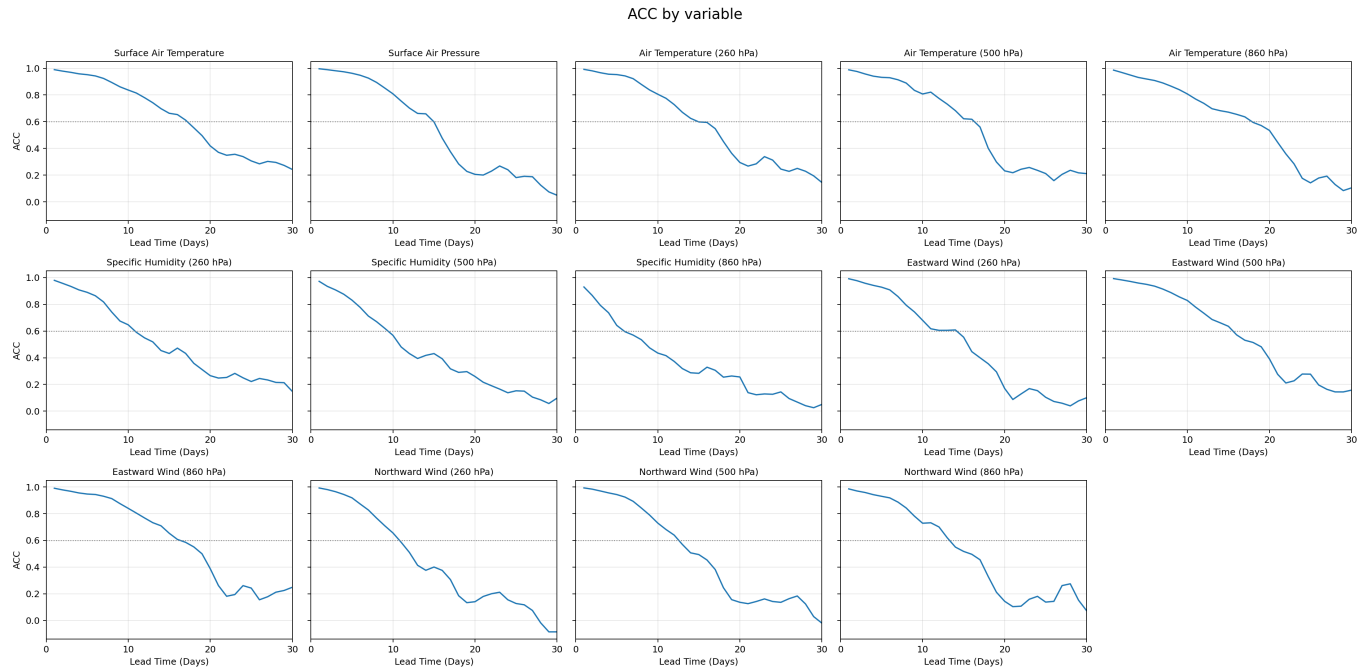
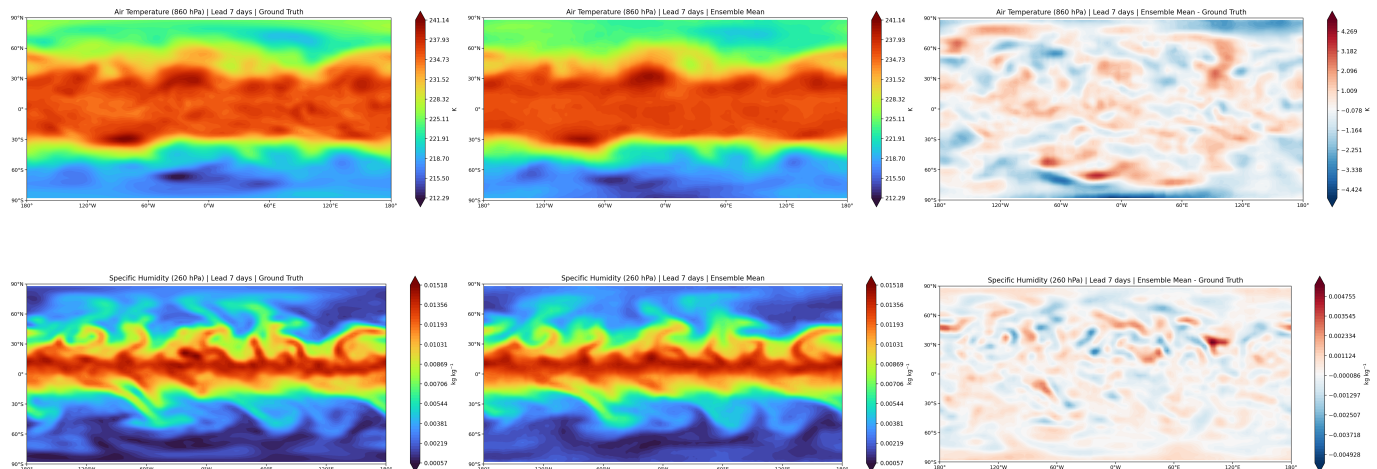


Figure 3: ACC plots for all variables

A field-plot comparison for four chosen variables is also shown below.



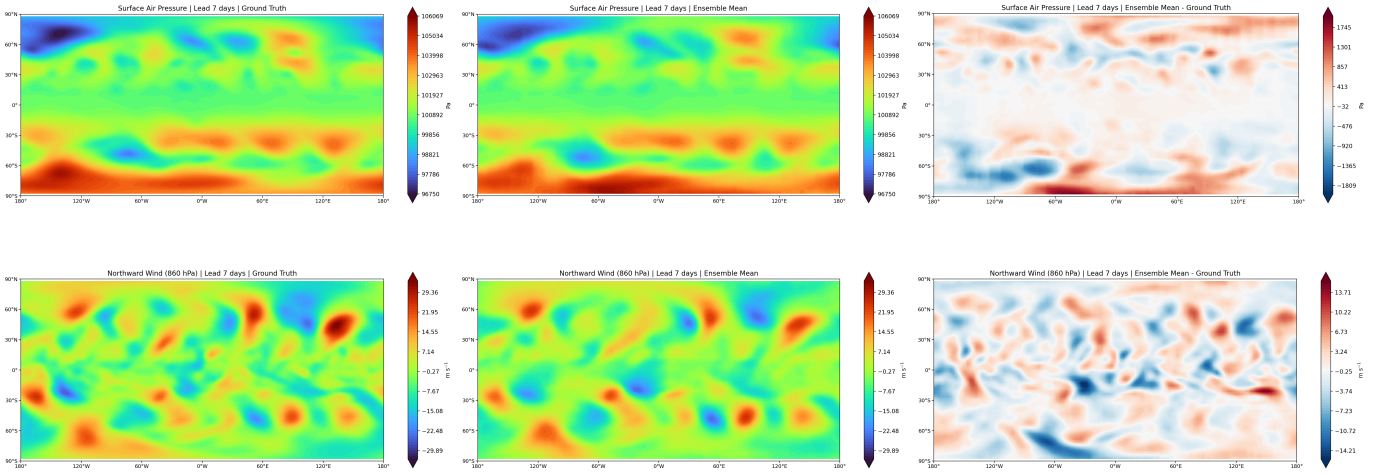


Figure 4: A qualitative evaluation of model's forecasting skill. Each row corresponds to one variable, with forecasts made at a lead time of 7-days: air-temperature at 860 hPa (row-1); specific humidity at 260 hPa (row-2); surface air pressure (row-3); northward wind at 860 hPa (row-4). In each row, the first plot represents the ground truth, the second plot represents the mean from 4 ensemble members, and the third plot represents the difference between the two: Forecast - Ground truth.

3 The GCM + AI Emulator Setup

3.1 Generating spinup state

We first generate a single spinup file by running the aquaplanet GCM for 3 years, and use the resulting saved state as the common initial condition for all subsequent runs.

As an example in our analyses, we consider the target region to be a fixed 3×3 box over northwest India, with the exact bounds give by 26.5°N – 32.1°N and 70.3°E – 75.9°E . Our observable is the final 7-day average regional-mean surface temperature over this box. The GCM is integrated with a 20-minute time step.

3.2 Direct Numerical Simulation (DNS)

The DNS ensemble consists of 400 walkers, all initialized from the same spinup state and separated by perturbations applied to the coefficients of spherical harmonics of surface pressure field. For our specific configuration, the perturbation amplitude is $\sqrt{2} \times 10^{-4}$. Each walker is then run for 37 days, and the observable is computed from the final 7-day regional-mean surface temperature.

3.3 Resampling with AI Emulator (AI+RES)

The AI+RES runs start from exactly the same 400 initial walkers and the same perturbation scheme as DNS. Resampling is done every $\tau = 5$ days using the schedule $C_k = [0, 0, 0, 1.8, 2.2, 2.6]$, so the first three 5-day intervals evolve without resampling pressure and the bias is introduced only later in the trajectory. For each of the 400 walkers, the AI emulator runs 100 ensemble members, calculates the mean of these members, and then estimates the score from the final 7-day regional-mean surface temperature; resampling is then applied as described in Section 3.5. Note that here the C_k schedule is a tunable hyperparameter and as such, its optimal value will differ from region to region. After trying out a number of different schemes, the present scheme seems to have worked best in providing a significant rightward shift in mean, while at the same time maintaining a decent variance in trajectories.

3.4 Results

Below in Figure 5, we show the evolution of each trajectory in the resampling simulation; the final day distribution of surface temperature for both the cases is plotted on the right.

In Figure 6(a), the distribution of our observable (7-day regional average surface temperature) is plotted for both – the DNS runs and AI+RES runs; we see a shift of ~ 3.49 K in the mean. Finally in Figure 6(b), the return time curves are plotted for both cases. We see that events with a return time of slightly higher than 10^4 years were captured in the resampling runs with only 400 walkers; whereas with the same number of walkers in DNS, only events with return times less than 1000 years could be captured.

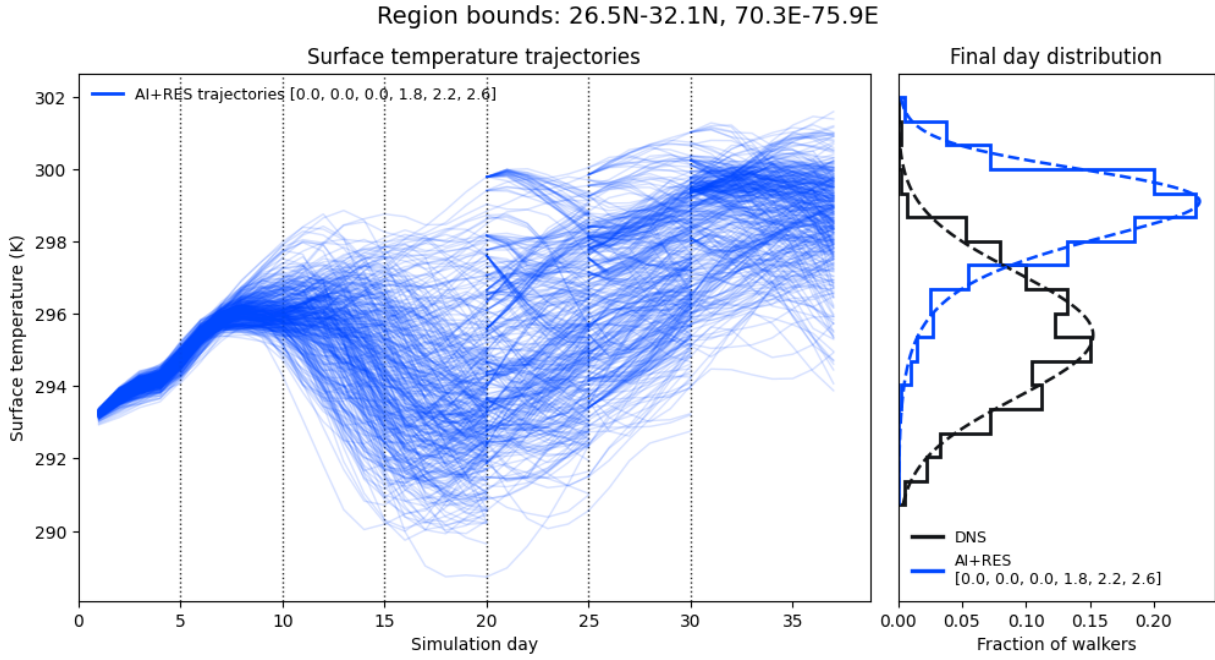
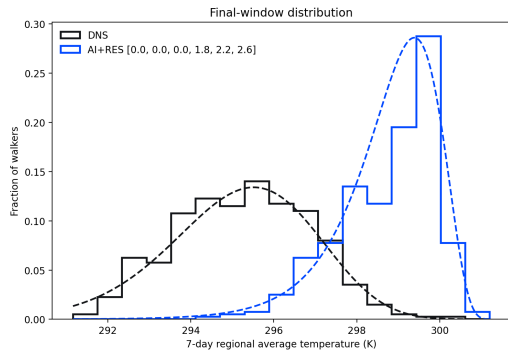
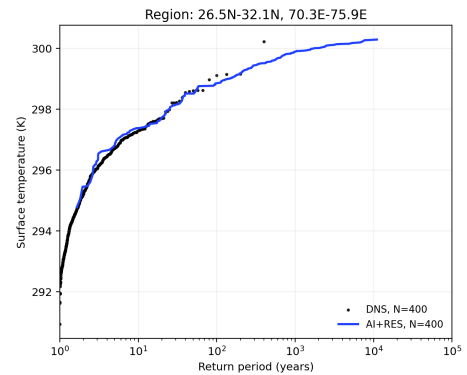


Figure 5: Evolution of each trajectory for the AI+RES setup; DNS trajectories not shown for the sake of maintaining clarity, but the final day DNS distribution can be seen on the right plot (black curve)

Region bounds: 26.5N-32.1N, 70.3E-75.9E (7-day regional average surface temperature distribution)



(a) Final 7-day regional average temperature distribution



(b) Return period curves

Figure 6: Analysis of observable temperature distributions [L] and return period curves [R] for both cases

3.5 Resampling algorithm

Algorithm 1 Ensemble simulation with AI Emulator Scoring and Pivotal Resampling [1]

Require: N walkers, K steps, AI emulator $\mathcal{E}$, Physics $\mathcal{M}$, parameter C_0

```

1: Initialize ensemble  $\{X_0^i\}_{i=1}^N$ 
2: for  $k = 0$  to  $K - 1$  do
3:   % AI Emulator Scoring
4:   for  $i = 1$  to  $N$  do
5:     Generate  $M$  forecasts from  $X_k^i$  to  $t_f$  using  $\mathcal{E}$ 
6:     Compute score  $\theta_k^i$  as ensemble mean over target observable
7:     Compute weight  $w_k^i = \exp(C_0\theta_0^i)$  if  $k = 0$  else  $\bar{w}_{k-1} \exp(C_k\theta_k^i - C_{k-1}\hat{\theta}_{k-1}^i)$ 
8:   end for
9:   for  $i = 1$  to  $N$  do
10:    Normalized weight  $\tilde{w}_k^i = Nw_k^i / \sum w_k^j$ 
11:     $N_k^i = \lfloor \tilde{w}_k^i \rfloor$ ,  $\delta_k^i = \tilde{w}_k^i - N_k^i$ 
12:   end for
13:   % Pivotal resampling
14:   Let  $U$  be active indices where  $0 < \delta_k^i < 1$ 
15:   while  $|U| > 1$  do
16:     Select  $a, b \in U$  and compute  $s = \delta_k^a + \delta_k^b$ 
17:     if  $s \leq 1$  then
18:       w.p.  $\delta_k^a/s$ :  $\delta_k^a \leftarrow s$ ,  $\delta_k^b \leftarrow 0$ 
19:       w.p.  $\delta_k^b/s$ :  $\delta_k^a \leftarrow 0$ ,  $\delta_k^b \leftarrow s$ 
20:     else if  $s > 1$  then
21:       w.p.  $\frac{1-\delta_k^b}{2-s}$ :  $\delta_k^a \leftarrow 1$ ,  $\delta_k^b \leftarrow s - 1$ 
22:       w.p.  $\frac{1-\delta_k^a}{2-s}$ :  $\delta_k^a \leftarrow s - 1$ ,  $\delta_k^b \leftarrow 1$ 
23:     end if
24:     if  $\delta_k^a \in \{0, 1\}$  then  $U \leftarrow U \setminus \{a\}$ ,  $N_k^a \leftarrow N_k^a + \delta_k^a$ 
25:     end if
26:     if  $\delta_k^b \in \{0, 1\}$  then  $U \leftarrow U \setminus \{b\}$ ,  $N_k^b \leftarrow N_k^b + \delta_k^b$ 
27:     end if
28:   end while
29:   % Now back to the GCM
30:   Form  $\{\hat{X}_k^j\}_{j=1}^N$  by cloning each  $X_k^i$  exactly  $N_k^i$  times
31:   for  $j = 1$  to  $N$  do
32:     Perturb  $\hat{X}_k^j$  and integrate:  $X_{k+1}^j = \mathcal{M}(\hat{X}_k^j)$ 
33:   end for
34: end for
Ensure: Final ensemble  $\{X_K^i\}_{i=1}^N$ 

```

References

- [1] Amaury Lancelin, Alex Wikner, Laurent Dubus, Clément Le Priol, Dorian S. Abbot, Freddy Bouchet, Pedram Hassanzadeh, and Jonathan Weare. AI-boosted rare event sampling to characterize extreme weather. *arXiv e-prints*, page arXiv:2510.27066, October 2025.
- [2] Xiaohui Zhong, Lei Chen, Hao Li, Roberto Buizza, Jun Liu, Jie Feng, Zijian Zhu, Xu Fan, Kan Dai, Jing jia Luo, Jie Wu, and Bo Lu. Fuxi-ens: A machine learning model for efficient and accurate ensemble weather prediction. *Science Advances*, 11(44):eadu2854, 2025.